UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

COMPUTACIÓN GRÁFICA E INTERACCIÓN HUMANO COMPUTADORA

PRACTICA 03: Modelado geométrico

Nombre de la alumna: Macias Niño Carmen Violeta

Nº de Cuenta: 318285450

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 02

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 08/03/2026

CALIFICACIÓN: _____

INTRODUCCION

En esta práctica se desarrolló un modelo tridimensional interactivo de un Pyraminx, que es una variante del cubo de Rubik con forma de tetraedro. Para su implementación se utilizaron OpenGL, GLFW, GLEW y la biblioteca matemática GLM.

El objetivo fue construir una figura tridimensional compuesta por cuatro caras triangulares subdivididas en nueve triángulos pequeños cada una. Además, se implementaron controles interactivos que permiten rotar, mover y acercar o alejar la figura utilizando el mouse y el teclado.

1. Inclusión de bibliotecas

```
#include <stdio.h>
#include <cmath>
#include <vector>

#include <glew.h>
#include <glfw3.h>

#include <glm.hpp>
#include <gtc/matrix_transform.hpp>
#include <gtc/type_ptr.hpp>

#include "Mesh.h"
#include "Shader.h"
#include "Window.h"
#include "Camera.h"
```

Explicación

Este bloque incluye todas las bibliotecas necesarias para el funcionamiento del programa.

- **stdio.h** permite realizar operaciones básicas de entrada y salida.
- **cmath** proporciona funciones matemáticas.
- **vector** permite manejar arreglos dinámicos de datos.

Las bibliotecas gráficas utilizadas son:

- **GLEW**: permite acceder a las funciones modernas de OpenGL.
- **GLFW**: gestiona la creación de ventanas y la captura de eventos de teclado y mouse.

Las bibliotecas de **GLM** se utilizan para operaciones matemáticas en gráficos 3D, como:

- Transformaciones
- Matrices
- Operaciones vectoriales

- **Mesh**: maneja la geometría de los objetos.
- **Shader**: controla los programas de sombreado.
- **Window**: administra la ventana y eventos.
- **Camera**: gestiona la cámara de la escena.

2. Variables globales

Código

```
GLfloat deltaTime = 0.0f;
GLfloat lastTime = 0.0f;

Window mainWindow;
Camera camera;
vector<Shader> shaderList;
```

Explicación

Se declaran variables globales utilizadas a lo largo del programa.

- **deltaTime** almacena el tiempo transcurrido entre frames.
- **lastTime** guarda el tiempo del frame anterior.

Estas variables permiten generar movimientos suaves y consistentes independientemente de la velocidad de ejecución.

También se crean los objetos principales:

- **mainWindow**: controla la ventana principal del programa.
- **camera**: representa la cámara que observa la escena.
- **shaderList**: almacena los shaders que se utilizarán para renderizar los objetos.

3. Variables de control del objeto

Código

```
static float rubikRotX = 25.0f;
static float rubikRotY = -35.0f;
static float rubikRotZ = 0.0f;

static glm::vec3 rubikPos = glm::vec3(0.0f, 0.0f, -4.5f);

static float rubikScale = 1.2f;
```

Explicación

Estas variables controlan la transformación del Pyraminx dentro del espacio tridimensional.

- **rubikRotX, rubikRotY, rubikRotZ** controlan la rotación del objeto sobre los ejes X, Y y Z.
- **rubikPos** almacena la posición del objeto en el espacio.
- **rubikScale** define el tamaño del objeto.

Estas variables se modifican mediante **interacciones del usuario** usando teclado o mouse.

4. Callback del scroll del mouse

Código

```
static void ScrollCallback(GLFWwindow* window, double xoffset, double
yoffset)
{
    gScrollY += (float)yoffset;
}
```

Explicación

Esta función captura el movimiento de la rueda del mouse.

El valor desplazado se acumula en la variable **gScrollY**, la cual posteriormente se utiliza para mover el objeto sobre el eje **Z**, permitiendo acercar o alejar el Pyraminx de la cámara.

5. Creación de shaders

Código

```
void CreateShaders()
{
    Shader* shader1 = new Shader();
    shader1->CreateFromFiles(vShader, fShader);
    shaderList.push_back(*shader1);
}
```

Explicación

Los **shaders** son programas que se ejecutan en la GPU para procesar los vértices y los colores de los objetos.

En este fragmento:

1. Se crea un objeto de tipo **Shader**.
2. Se cargan los archivos del shader de vértices y fragmentos.
3. Se almacena el shader en un vector para su uso posterior.

6. Estructura de almacenamiento de caras

Código

```
struct FaceMeshes
{
    Mesh* blackMesh;
    Mesh* colorMesh;
    glm::vec3 faceColor;
};
```

Explicación

Esta estructura se utiliza para almacenar la información de cada cara del Pyraminx.

Cada cara contiene:

- **blackMesh**: triángulos negros que representan las separaciones.
- **colorMesh**: triángulos de color que forman la superficie visible.
- **faceColor**: color específico de la cara.

Esto permite renderizar las caras de forma organizada.

7. Generación de triángulos

Código

```
static void PushTri(vector<GLfloat>& verts, vector<unsigned int>& idx,
    const glm::vec3& a, const glm::vec3& b, const glm::vec3& c)
```

Explicación

Esta función agrega un triángulo a los arreglos de vértices e índices.

Cada triángulo está formado por **tres vértices**, los cuales se almacenan en el vector de vértices.
Posteriormente se agregan los índices correspondientes para que OpenGL pueda dibujar el triángulo.

Este método facilita la construcción dinámica de la geometría del objeto.

8. Cálculo de coordenadas baricéntricas

Código

```
static glm::vec3 BaryPoint(const glm::vec3& A,
                           const glm::vec3& B,
                           const glm::vec3& C,
                           float u,
```

```
                                float v)
{
    return A * (1.0f - u - v) + B * u + C * v;
}
```

Explicación

Esta función calcula un punto dentro de un triángulo usando **coordenadas baricéntricas**.

Estas coordenadas permiten dividir un triángulo en subtriángulos más pequeños, lo cual es fundamental para generar las **9 subdivisiones del Pyraminx**.

9. Construcción de las caras del Pyraminx

```
static void BuildFaceMeshes(...)
```

Explicación

Esta función genera una cara triangular subdividida en múltiples triángulos pequeños.

El proceso consiste en:

1. Calcular la **normal de la cara**.
2. Subdividir el triángulo en **9 subtriángulos**.
3. Crear dos versiones de cada triángulo:
    - uno negro (bordes)
    - uno de color ligeramente reducido.

Esto produce el efecto visual de **líneas negras entre piezas**, simulando un rompecabezas real.

10. Construcción completa del Pyraminx

Código

```
void CrearPyraminx()
```

Explicación

En esta función se define la geometría del tetraedro que forma el Pyraminx. Se definen los cuatro vértices principales:

```
glm::vec3 V0
glm::vec3 V1
glm::vec3 V2
glm::vec3 V3
```

Estos vértices generan las cuatro caras triangulares del tetraedro.

Posteriormente se llama a **BuildFaceMeshes** para cada cara, asignando colores distintos:

- Amarillo
- Rojo
- Verde
- Morado

11. Inicialización del programa

Código

```
mainWindow = Window(900, 700);
mainWindow.Initialise();
```

Explicación

Se crea una ventana de **900 x 700 píxeles** donde se renderizará la escena.

Luego se inicializa OpenGL y se configura el entorno gráfico.

12. Activación del buffer de profundidad

Código

```
glEnable(GL_DEPTH_TEST);
```

Explicación

El **Depth Test** permite que OpenGL determine qué objetos están delante y cuáles están detrás.

Sin esta función los objetos se dibujarían en orden incorrecto, generando errores visuales en la escena 3D.

13. Transformaciones del modelo

Código

```
baseModel = glm::translate(baseModel, rubikPos);
baseModel = glm::rotate(baseModel, glm::radians(rubikRotX),
glm::vec3(1,0,0));
baseModel = glm::rotate(baseModel, glm::radians(rubikRotY),
glm::vec3(0,1,0));
baseModel = glm::rotate(baseModel, glm::radians(rubikRotZ),
glm::vec3(0,0,1));
```

```
baseModel = glm::scale(baseModel, glm::vec3(rubikScale));
```

Explicación

Estas operaciones transforman el objeto dentro del espacio tridimensional:

1. **Translate** mueve el objeto en el espacio.
2. **Rotate** aplica rotaciones en los ejes X, Y y Z.
3. **Scale** ajusta el tamaño del modelo.

Estas transformaciones se aplican antes de renderizar la figura.

14. Renderizado de la figura

Código

```
for (size_t f = 0; f < pyrFaces.size(); f++)
{
    pyrFaces[f].blackMesh->RenderMesh();
    pyrFaces[f].colorMesh->RenderMesh();
}
```

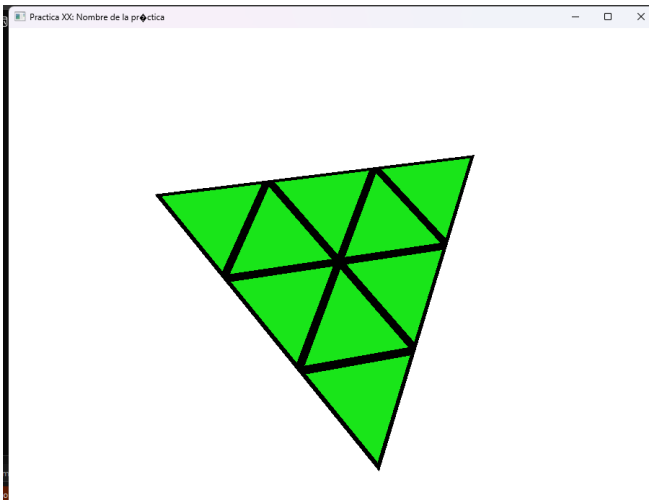
Explicación

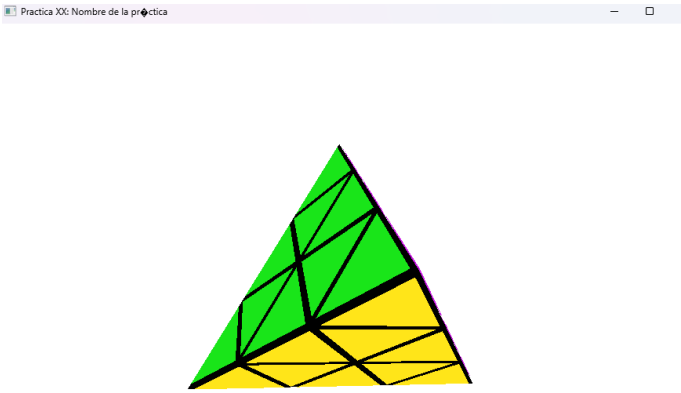
Este ciclo recorre todas las caras del Pyraminx y dibuja:

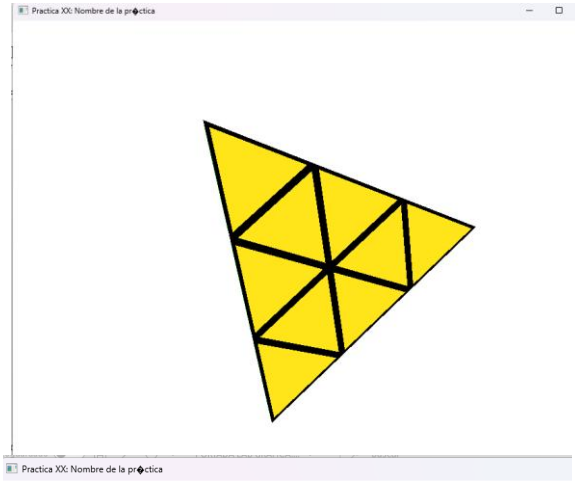
1. Primero los triángulos negros (bordes).
2. Después los triángulos de color.

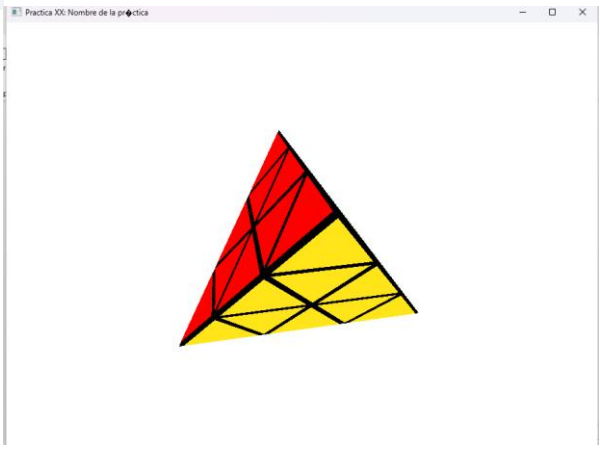
El uso de RenderMesh() permite dibujar los triángulos utilizando GL_TRIANGLES, lo cual es necesario para que se rendericen correctamente las subdivisiones.

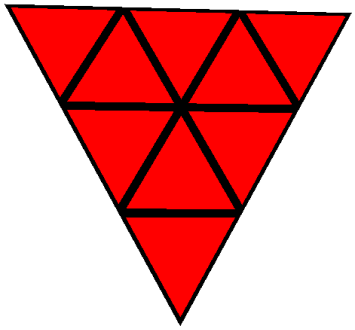
FUNCIONALIDA D

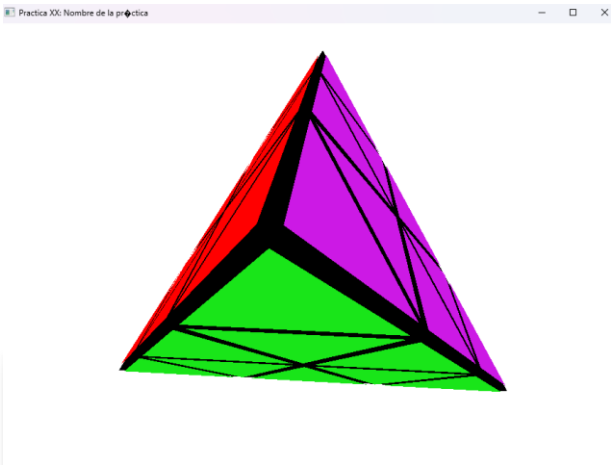




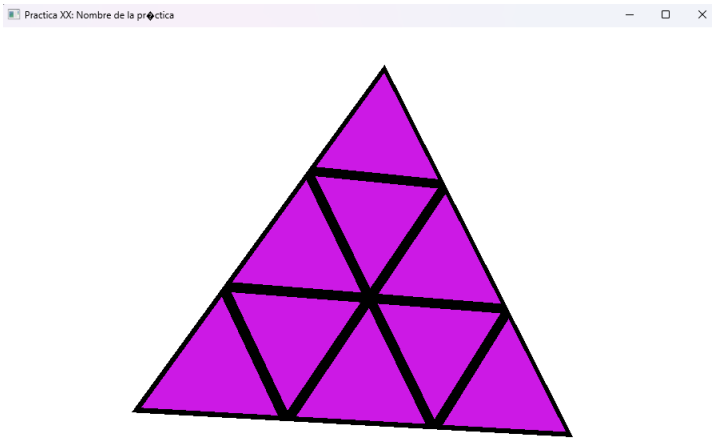

Practica XX: Nombre de la pr□ctica
Practica XX: Nombre de la pr□ctica

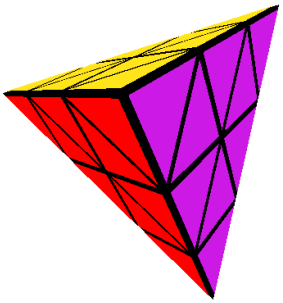

Practica XX: Nombre de la pr□ctica

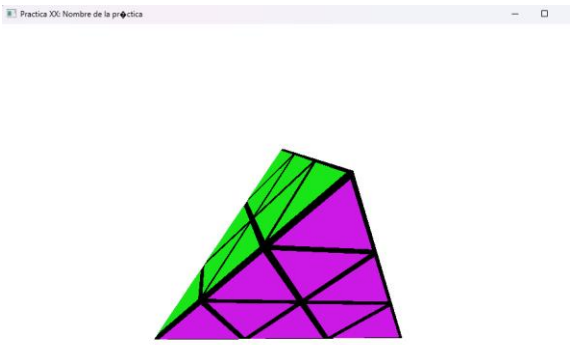


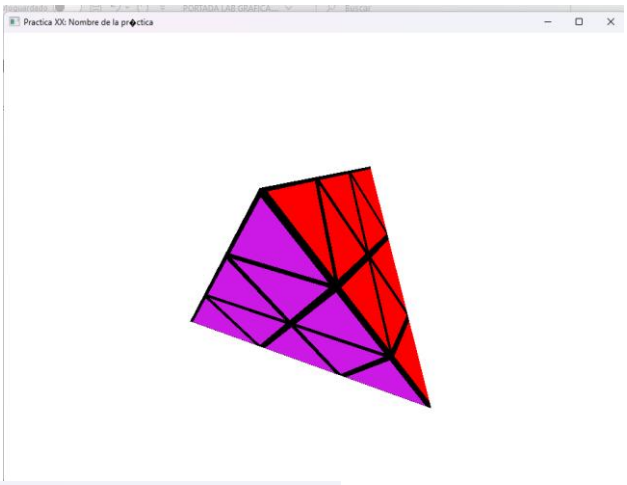

Practica XX: Nombre de la pr□ctica

Practica XX: Nombre de la pr□ctica


Practica XX: Nombre de la pr□ctica







Conclusión

Con esta práctica pude entender mejor cómo construir una figura tridimensional a partir de primitivas simples, en este caso triángulos, y cómo a partir de ellos se puede formar una estructura más compleja como el Pyraminx. También me ayudó a comprender mejor el uso de transformaciones en OpenGL, especialmente traslación, rotación y escala, además de la importancia de organizar correctamente los vértices, índices y colores para que la figura se renderice como se espera. Otra parte importante fue implementar la interacción con el mouse y el teclado, porque eso permitió que el modelo no fuera estático, sino manipulable en el espacio. En general, la práctica me sirvió para relacionar la parte matemática de la geometría con su representación visual en un entorno gráfico.